AI TECHNICAL DEBT PRIORITIZER

Core ML, AI and Algorithms

How the system learns, ranks, explains and plans technical-debt fixes

Purpose. This document explains every machine-learning, AI and algorithmic component of the AI Technical Debt Prioritizer (TDP): how the dataset is mined from public repositories, how every feature is computed, how labels are produced without leaking the future into the past, how the learning-to-rank model is trained and evaluated, how scores are calibrated and explained, and how the final, budget-aware fix order is computed.

Companion documents. Document 2 covers the software and DevOps work (services, data model, CI/CD, deployment, security). Document 3 lists every change made to the original idea and defends each one.

Note on figures. Charts marked *illustrative* use synthetic values to show what an output or metric looks like. They are not results. Real numbers come from running the evaluation protocol in Section 8.

Contents

1. Problem Formulation . . . 4

1.1 Unit of analysis . . . 4

1.2 Formal statement . . . 4

1.3 What the output looks like . . . 4

2. End-to-End ML Pipeline . . . 5

3. Dataset Construction . . . 6

3.1 Repository selection . . . 6

3.2 Snapshots and rows . . . 6

3.3 Mining strategy . . . 6

4. Feature Engineering . . . 7

4.1 Family S: static structure (Tree-sitter) . . . 7

4.2 Family E: evolution (git history) . . . 7

4.3 Family O: ownership . . . 8

4.4 Family D: defect history . . . 8

4.5 Family I: impact and coupling (NetworkX) . . . 8

4.6 Family T: test signals . . . 9

4.7 Family Q: self-admitted technical debt (SATD) . . . 9

4.8 Clone detection algorithm . . . 9

4.9 Normalisation . . . 9

5. Labels: Measuring Future Maintenance Pain . . . 10

5.1 Step 1: identifying bug-fix commits . . . 10

5.2 Step 2: SZZ, finding where the bug actually was . . . 10

5.3 Step 3: the latent-defect rule . . . 11

5.4 Step 4: from events to a graded relevance label . . . 11

6. The Ranking Model . . . 12

6.1 Why learning-to-rank . . . 12

6.2 Ranking quality: DCG and NDCG . . . 12

6.3 LambdaMART . . . 12

6.4 Hyperparameter search . . . 13

6.5 Optional monotone constraints . . . 13

6.6 Baselines . . . 13

7. Feature Selection and Model Hygiene . . . 13

8. Evaluation Protocol . . . 14

8.1 Two splits, both honest . . . 14

8.2 Metrics . . . 15

8.3 Statistical comparison . . . 16
8.4 Ablation study . . . 16
8.5 External sanity check with PROMISE . . . 16
9. Calibration: Giving the Score a Meaning . . . 16
9.1 Priority (0 to 100): a relative position . . . 17
9.2 Risk (0 to 100%): an absolute probability . . . 17
10. Explainability with SHAP . . . 17
10.1 Shapley values . . . 17
10.2 From SHAP values to readable reasons . . . 18
10.3 Global explanations . . . 18
11. Debt Types and SATD Classification . . . 18
11.1 Debt-type assignment . . . 18
11.2 SATD detection . . . 19
12. Effort-Aware Prioritisation and the Fix Plan . . . 19
12.1 Expected benefit . . . 19
12.2 Remediation cost . . . 20
12.3 Budgeted selection: 0/1 knapsack . . . 20
12.4 Ordering the plan: dependency-aware scheduling . . . 21
12.5 What-if simulator . . . 21
13. Generative AI Layer: Grounded Fix Advice . . . 22
13.1 Inputs (grounding) . . . 22
13.2 Output schema . . . 22
13.3 Validation (no unchecked text reaches the user) . . . 22
13.4 Model choice, cost and privacy . . . 22
13.5 Evaluating the advice . . . 23
14. Learning from Feedback . . . 23
15. Extensions and Threats to Validity . . . 23
15.1 Extensions . . . 23
15.2 Threats to validity . . . 23
16. References . . . 24

1. Problem Formulation

Static-analysis tools answer the question *"what is wrong with this code?"*. A team with limited time needs the answer to a different question: *"which problems should we fix first, given what each one will cost us if we leave it and what it will cost to fix?"*. TDP is built to answer the second question.

1.1 Unit of analysis

The unit is a **source file (module) at a snapshot time T**. Files are the natural unit for git history (every commit records which files it touched), for import graphs (Python and Java import modules, not functions) and for ownership. Function-level analysis is a later extension and is discussed in Section 15.

1.2 Formal statement

For a repository *r* and a snapshot time *T*, let *F(r, T)* be the set of source files that exist at *T*. For every file *f* we compute a feature vector $x_f(T)$ using **only information available at or before T**. We then observe what actually happens to the file in the horizon window *(T, T + H]* (H = 12 months by default) and summarise it as a graded relevance label y_f in {0, 1, 2, 3, 4}.

The learning problem is to find a scoring function *s(x)* such that, within each repository-snapshot, sorting files by $s(x_f)$ reproduces the ordering by y_f as closely as possible, with the most weight placed on getting the top of the list right. This is a **learning-to-rank** problem in which each (repository, snapshot) pair is one query and its files are the documents to rank.

The ranked list is then converted into a **fix plan**: each file receives a calibrated risk probability, an impact multiplier and an estimated remediation cost, and a knapsack optimiser picks the set of fixes that gives the largest expected benefit within the team's time budget. A dependency-aware scheduler orders that set.

> **The one-sentence version**
> TDP predicts which files are most likely to cause maintenance pain in the next year, explains why, and tells the team which of them give the best return per hour of fixing.

1.3 What the output looks like

For every file the system reports a priority score with a defined meaning, the top reasons in plain language, the debt type and a recommended action. An example:

Example report entry (illustrative)

```
src/payments/processor.py      Priority 92 / 100  (Critical, top 3% of files in this repo)
  Estimated chance of a bug fix in the next 12 months: 64%
  Why:
    1. Changes very often: 47 commits in the last 6 months (top 4% in repo)
    2. Bug history: 6 earlier bugs traced to code in this file
    3. Wide impact: 23 modules depend on it, directly or transitively
    4. No test file imports this module
  Debt type: Test debt + Change-coupling debt
  Estimated fix effort: 6 to 9 hours        Return on effort: rank 2 of 412
  Plan: in the 40-hour plan, step 1 (fix before billing/invoice.py, which depends on it)
```

2. End-to-End ML Pipeline

Figure 1 shows the complete pipeline. The top row builds the dataset. The middle row produces labels from the future window and is used only during training and evaluation. The bottom row is the inference path that runs when a user analyses a repository.

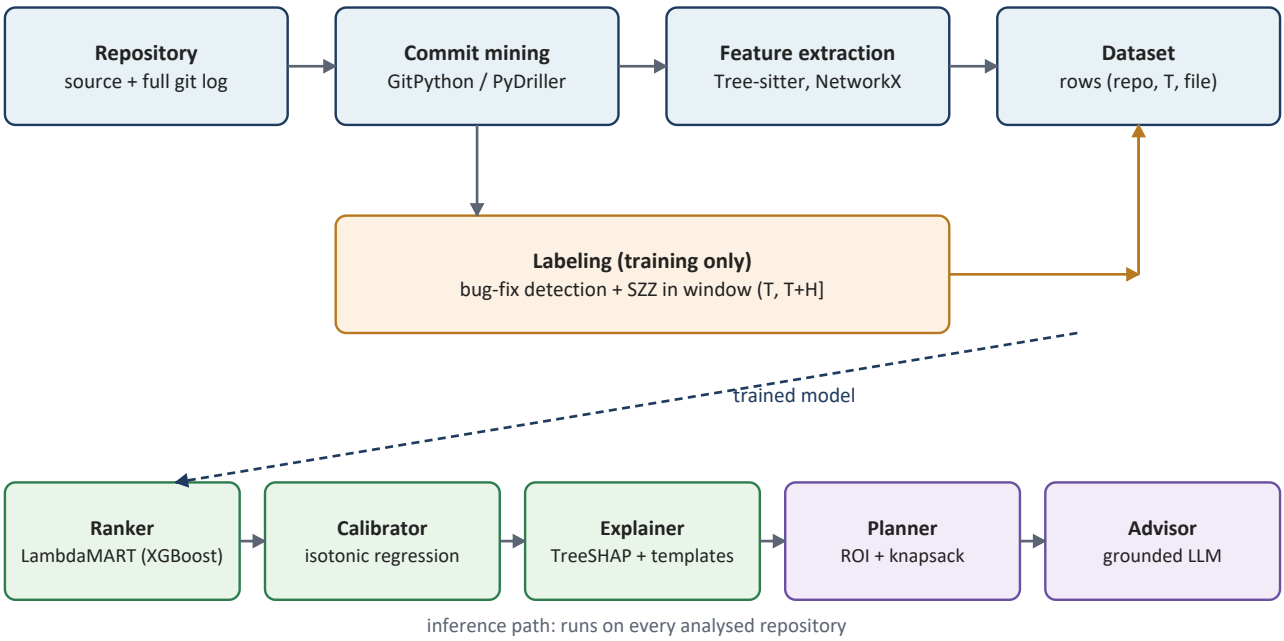


Figure 1. TDP pipeline. Blue: data construction. Amber: label construction (never used at inference). Green: learned components. Purple: algorithmic and generative layers.

Stage	Input	Output	Key technique
Commit mining	git repository	per-commit file changes, authors, dates, messages	git log --numstat parsing, PyDriller
Feature extraction	files at snapshot T + history up to T	about 40 numeric features per file	Tree-sitter ASTs, graph algorithms, decay weighting
Labeling	history in (T, T+H]	graded relevance 0 to 4 per file	bug-fix detection, SZZ, latent-defect rule
Ranker	features, query groups	ranking score s(x)	LambdaMART (XGBoost rank:ndcg)
Calibrator	s(x)	P(latent bug fixed within H)	isotonic regression
Explainer	model + x	per-feature contributions, reasons	TreeSHAP, family grouping
Planner	risk, impact, cost, graph	budgeted, ordered fix plan	0/1 knapsack DP, SCC + topological sort
Advisor	reasons + code excerpt	concrete refactoring steps	LLM with schema and AST validation

Table 1. Pipeline stages at a glance.

3. Dataset Construction

3.1 Repository selection

The dataset is mined from public GitHub repositories. Selection criteria keep the labels meaningful:

- **Language:** Python first (one Tree-sitter grammar, one import resolver). Java is the optional second language.
- **History:** at least 3 years and 1,000 commits, so several snapshots each have a full 12-month label window.
- **Size:** at least 80 source files, so a ranking within the repository is meaningful.
- **Bug tracking:** GitHub issues with a bug-type label (for example *bug*, *type: bug*, *kind/bug*) and commit messages that reference issues. This gives high-precision bug-fix links.
- **Activity:** commits in the last 12 months; archived projects are excluded.
- **Diversity:** a mix of libraries, web frameworks, CLI tools and data tools, from different organisations, so that cross-project evaluation is honest.
- **License:** permissive open-source licenses only; only derived metrics are stored, never copied source.

A target of **20 to 30 repositories** is enough. Candidates of the right shape include projects such as *requests*, *flask*, *httpx*, *scrapy*, *black*, *poetry*, *celery*, *sqlalchemy*, *pydantic*, *fastapi* and *rich*; each must be checked against the criteria before use.

3.2 Snapshots and rows

Each repository is sampled at several snapshot times, every 6 months, starting once the project has at least one year of history and stopping 12 months before the newest commit (the last label window must be complete). One row is one **(repository, snapshot, file)** triple.

With 25 repositories, about 6 snapshots each and a few hundred files per snapshot, the dataset holds roughly 25 x 6 x 300 = 45,000 rows. That is plenty for gradient-boosted trees, and every row is grounded in real project history.

> **Why several snapshots per repository?**
> One snapshot per repository would give only 25 queries for a ranking model. Several snapshots give 150 or more queries and show how the same file's risk evolves. Rows from the same repository are correlated, so all splits are done by repository and time, never by random row (Section 8).

3.3 Mining strategy

The miner walks the full history once and records, for every commit: hash, author (normalised by email and a *.mailmap* file), timestamp, message, parent hashes and, for each touched file, lines added and deleted plus the rename source if any. Renames are followed (*git log -M*) so a file keeps its history when moved. Merge commits are skipped for churn (their changes are already counted in the merged branch) and bulk commits that touch more than 50 files (formatting runs, license headers, vendored code) are flagged and excluded from churn and co-change statistics.

4. Feature Engineering

Features are organised into seven families. Grouping matters for three reasons: the ablation study measures each family's contribution, SHAP explanations are summarised per family, and debt types are derived from which family dominates. Every feature is computed from data at or before *T*.

4.1 Family S: static structure (Tree-sitter)

Tree-sitter parses each file into a concrete syntax tree. Queries over the tree extract functions, classes, branches and comments for any language with a grammar, which keeps the extractor language-agnostic.

- **SLOC**: non-blank, non-comment lines.
- **Cyclomatic complexity (McCabe)** per function: M = E - N + 2P on the control-flow graph, computed in practice as 1 + the number of decision points (*if, elif, for, while, except, case*, boolean operators *and/or*, conditional expressions, comprehension *if* clauses). File features: sum, mean and maximum over functions.
- **Cognitive complexity** (optional): like cyclomatic complexity, but nesting increases the penalty, which tracks human difficulty more closely.
- **Maximum nesting depth**, **number of functions**, **number of classes**, **longest function** (lines), **parameter count** (maximum).
- **Duplication ratio**: fraction of the file's tokens that belong to a clone (see 4.8).

4.2 Family E: evolution (git history)

Raw lifetime counts treat a change from five years ago the same as one from last week. TDP keeps raw counts for reference and adds recency-weighted versions. With change i at age a_i days before T:

$$churn_w(f) = \sum_i (added_i + deleted_i) \cdot e^{-\lambda a_i}, \quad \lambda = \ln 2 / half\text{-}life$$

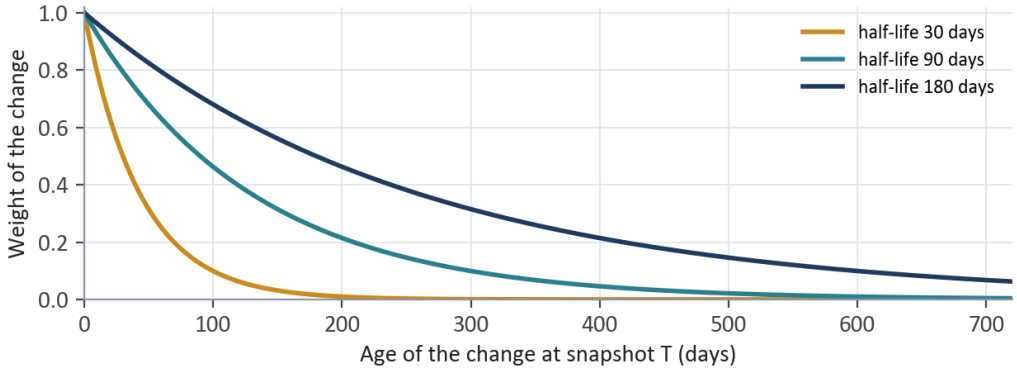


Figure 2. Exponential decay weights. The default half-life is 90 days: a change made 3 months before T counts half as much as one made at T. The half-life is tuned on validation data.

- **Commits** touching the file: lifetime, last 90 days, last 180 days, and the decayed count.
- **Churn**: lines added + deleted (decayed and raw), and **relative churn** = churn / SLOC. Relative churn is a stronger defect predictor than absolute churn (Nagappan and Ball, 2005).
- **Age** (days since creation) and **staleness** (days since last change).
- **Complexity trend**: slope of a least-squares line through the file's total complexity at its last 10 revisions. A positive slope means the debt is growing.
- **Burstiness**: coefficient of variation of the gaps between commits; files edited in frantic bursts are often fire-fighting sites.

4.3 Family O: ownership

Bird et al. (2011) showed that files with many *minor* contributors have more defects than files with a clear owner. For each file, let p_a be the share of its changes made by author a:

- **Contributors**: number of distinct authors.
- **Owner share**: $\max_a p_a$.
- **Minor contributors**: authors with $p_a < 5\%$.
- **Author entropy**: $H = -\sum_a p_a \log_2 p_a$. High entropy means knowledge is spread thinly.
- **Owner activity**: whether the top author has committed anywhere in the repository in the last 90 days. If the owner has left, the file carries knowledge debt.

4.4 Family D: defect history

These features count bug-related events **before T**: bug-fix commits that touched the file, and the number of bugs that SZZ (Section 5) traces back to lines in the file. They are legitimate predictors because the label only counts events after T. Past bugs are known at prediction time; future bugs are not.

> **Why this is no longer leakage**
> The original idea used bug-fix counts as a feature and also derived the label from bug activity over the same period, so the model could simply copy its input. Here the feature window ends at T and the label window starts after T, so the model must *predict* future bugs from past signals. Document 3 (change C1) discusses this in detail.

4.5 Family I: impact and coupling (NetworkX)

Two graphs are built at every snapshot.

Import graph G_{imp}

Directed edge a to b when module a imports module b. Python imports are resolved to files using the package layout (absolute and relative imports, *__init__.py* re-exports); imports of third-party packages are dropped. Features:

- **Fan-in** (direct dependents) and **fan-out** (direct dependencies).
- **Transitive dependents**: the size of the set of modules that can reach f in G_{imp}, computed with one reverse breadth-first search per file, or once per strongly connected component on the condensed graph.
- **PageRank** (damping 0.85) on the reversed graph: a module is important if important modules depend on it.
- **Betweenness centrality**: how often the module lies on shortest dependency paths (bridges between subsystems).
- **Cycle membership**: size of the strongly connected component containing f (cycles are architecture debt).

Co-change graph G_{co}

Imports miss many real dependencies: dynamic imports, plugin registries, configuration files, templates, SQL migrations, and code in other languages. Files that keep changing together are coupled whether or not they import each other (Gall et al., 1998). Let C_f be the set of non-bulk commits touching f before T. For every pair:

$$support(a, b) = |C_a \cap C_b|, \quad confidence(a \to b) = |C_a \cap C_b| \,/\, |C_a|$$

An edge is kept when support is at least 5 and confidence is at least 0.5. Features: number of strong co-change partners, maximum confidence, and **hidden coupling**: the number of strong partners that are not connected in G_{imp}. Hidden coupling is a direct measure of architecture debt that static tools cannot see. Pair counting uses a sparse file-by-commit matrix M: $M^T M$ gives all supports in one sparse multiplication.

4.6 Family T: test signals

Measuring real line coverage would require building and running the test suite of every repository at every snapshot. That is slow, often fails on old snapshots, and means executing untrusted code. TDP uses static proxies that are almost as informative for ranking:

- **Has a matching test file** (for *pkg/foo.py*: *tests/test_foo.py*, *test_foo.py*, *foo_test.py*).
- **Test importers**: number of test files that import the module (from G_{imp} restricted to test files).
- **Test co-change**: fraction of the file's commits that also touched a test file. Code that is changed without touching tests is a test-debt signal.
- **Package test ratio**: test SLOC / source SLOC within the file's package.

For the 2 or 3 demo repositories, real coverage is measured with *coverage.py* in a sandbox and shown in the dashboard. It also validates the proxies: the correlation between proxies and real coverage is reported.

4.7 Family Q: self-admitted technical debt (SATD)

Developers often admit debt in comments: *"TODO: this is a hack until the v2 API lands"*. Comments are extracted with Tree-sitter, classified (Section 11), and summarised as **SATD count**, **SATD density** per 1,000 SLOC and **SATD age** (age of the oldest SATD comment, from blame).

4.8 Clone detection algorithm

Duplication is detected with token-based fingerprinting, which is fast and robust to renamed variables:

1. Tokenise each file with Tree-sitter. Replace identifiers with *ID* and literals with *LIT* so that copy-paste-rename clones match (Type-2 clones).
2. Slide a window of k = 50 tokens and compute a rolling Rabin-Karp hash for each window.
3. Keep a subset of fingerprints with **winnowing** (the minimum hash in each window of w hashes), which guarantees that any match of at least w + k - 1 tokens is detected while storing far fewer hashes.
4. Group equal fingerprints across the repository in a hash table, then extend matches to maximal regions.
5. Duplication ratio of a file = tokens covered by any clone region / total tokens.

4.9 Normalisation

Raw values differ by orders of magnitude between projects: 40 commits a month is heavy in a small library but quiet in a large framework. Two transformations are applied:

- **log1p** for heavy-tailed counts (churn, SLOC, dependents), which stabilises tree splits.
- **Within-snapshot percentile rank**: each feature is also expressed as its percentile among files in the same repository-snapshot. The model sees both raw and percentile forms and learns which generalises across projects; in cross-project defect prediction the relative forms usually transfer better.

Family	Features (examples)	Source	Debt it reveals
S Static	SLOC, CC sum/mean/max, cognitive complexity, nesting, duplication ratio	Tree-sitter	Complexity, duplication
E Evolution	decayed churn, relative churn, commit counts, age, complexity trend, burstiness	git log	Change-prone hotspots
O Ownership	contributors, minor contributors, owner share, author entropy, owner active	git log, .mailmap	Knowledge debt
D Defect history	prior bug-fix commits, prior SZZ-traced bugs (before T)	git + issues + SZZ	Chronic defect sites
I Impact	fan-in/out, transitive dependents, PageRank, betweenness, SCC size, hidden coupling	NetworkX	Architecture debt, blast radius
T Tests	matching test file, test importers, test co-change, package test ratio	files + graph	Test debt
Q SATD	SATD count, density, age	comments + classifier	Admitted debt

Table 2. Feature families. About 40 features in total, plus percentile versions.

5. Labels: Measuring Future Maintenance Pain

5.1 Step 1: identifying bug-fix commits

A commit is classified as a bug fix by three sources, used in order of precision:

1. **Issue linkage (highest precision).** The message references an issue (*#123*, *fixes #123*, *closes GH-123*) or the commit belongs to a merged pull request that closes an issue, and that issue carries a bug-type label. Data comes from the GitHub REST/GraphQL API and is cached.
2. **Keyword heuristic (fallback).** A regular expression on the message for *fix, bug, defect, crash, error, regression, fault, incorrect, wrong*, excluding false-positive patterns such as *fix typo*, *fix lint*, *fix docs*, *prefix*, *suffix*, and commits that only touch documentation or test files.
3. **Commit classifier (optional).** A TF-IDF + logistic-regression model on commit messages, trained on the commits labelled by source 1. It improves recall on projects with few issue links.

Quality check: 200 commits are sampled and labelled by hand, and the precision and recall of the detector are reported. Label noise is a known threat in defect prediction, so measuring it is part of the method.

5.2 Step 2: SZZ, finding where the bug actually was

A bug-fix commit often touches test files, changelogs and unrelated clean-up as well as the buggy code. Counting every touched file as buggy adds noise. The SZZ algorithm (Sliwerski, Zimmermann and Zeller, 2005) narrows this down to the lines that were repaired:

1. For the fix commit c_{fix}, take the lines it deleted or modified in each source file.
2. Ignore whitespace-only, comment-only and import-reordering lines, and lines in test files.
3. Run *git blame* on the parent of c_{fix} for those lines; the commit that last touched each line is a **bug-inducing candidate**.
4. Discard candidates made after the bug was reported (from the issue's creation date), and candidates that are bulk or pure-refactoring commits, following the refinements of Kim et al. (2006) and later RA-SZZ work.

PyDriller provides a tested implementation (*get_commits_last_modified_lines*) that the miner wraps.

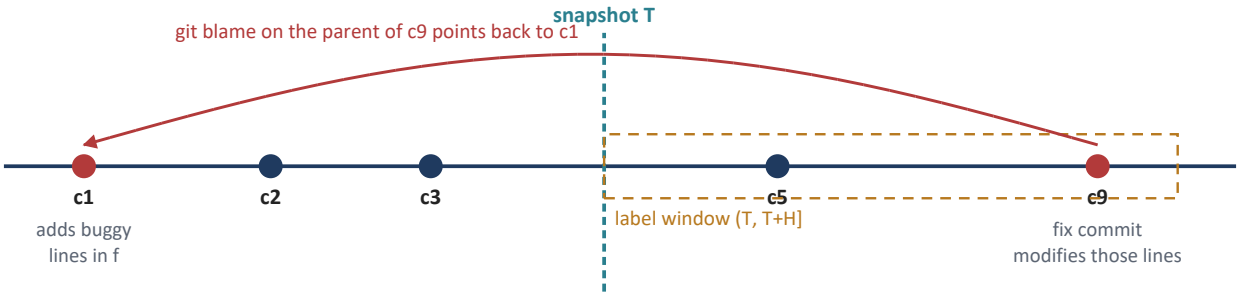


Figure 3. SZZ in one picture. The buggy lines were introduced by c1 before T and repaired by c9 after T, so file f had a latent defect at T. This event counts towards f's label for snapshot T.

5.3 Step 3: the latent-defect rule

A file *f* is credited with a **latent defect** at snapshot *T* when a bug-fix commit in *(T, T+H]* repairs lines in *f* whose SZZ-identified inducing commit is at or before *T*. In words: the bug already existed in the code at the moment of prediction, and it was found and fixed within the next year. This is the cleanest available definition of "this file contained a defect we could have prevented". It mirrors the post-release defect definition used in classic defect-prediction datasets.

5.4 Step 4: from events to a graded relevance label

Maintenance pain has two parts: defects (risk) and change effort (cost of keeping the file working). For each file and snapshot:

$$P_f = B_f + \gamma \cdot \log(1 + C_f / \text{median}_r(C))$$

where B_f is the number of latent defects fixed in the window, C_f is the file's churn in the window, the median is taken over the repository-snapshot (so churn is relative to the project's normal pace), and $\gamma = 0.25$ so that defects dominate. Ranking models need small integer grades, so *P* is converted into a relevance grade:

Grade	Rule (within the repository-snapshot)	Meaning
0	P = 0	Nothing happened: no latent defect, no notable churn
1	P > 0, lowest 50% of non-zero P	Some maintenance activity
2	50th to 80th percentile of non-zero P	Clear maintenance burden
3	80th to 95th percentile of non-zero P	Heavy burden, usually with defects
4	top 5% of non-zero P	Critical: the files that hurt most

Table 3. Relevance grades used for training the ranker.

A **sensitivity analysis** re-trains with γ in {0, 0.1, 0.25, 0.5} and H in {6, 12} months and reports how much the top-10 lists change (Kendall's tau and top-10 overlap). If conclusions only hold for one setting, the report says so.

Two auxiliary targets are kept on every row: the binary *has_latent_defect* (for calibration, Section 9) and the raw B_f (for effort-aware metrics, Section 8).

6. The Ranking Model

6.1 Why learning-to-rank

The product shows a ranked list, and users act on the top of it. A model trained to minimise squared error or log-loss spends most of its capacity on the many unremarkable files in the middle, and its errors are not weighted by position. Learning-to-rank optimises the ordering directly, with the largest penalty for mistakes near the top.

Approach	Loss looks at	Example	Fit for TDP
Pointwise	each file alone (regress or classify y)	XGBoost regressor, Random Forest	Baseline only
Pairwise	pairs of files in the same query: is i above j?	RankNet, XGBoost rank:pairwise	Good
Listwise	the whole ranked list, weighted by position	LambdaMART, XGBoost rank:ndcg	Chosen

Table 4. Families of learning-to-rank methods.

6.2 Ranking quality: DCG and NDCG

For a ranked list where the file at position i has grade rel_i:

$$DCG@k = \sum_{i=1..k} (2^{rel_i} - 1) / log_2(i + 1), \quad NDCG@k = DCG@k / IDCG@k$$

IDCG is the DCG of the perfect ordering, so NDCG is between 0 and 1. The exponential gain rewards putting grade-4 files on top, and the logarithmic discount makes position 1 matter much more than position 10.

6.3 LambdaMART

LambdaMART (Burges, 2010) combines gradient-boosted regression trees (MART) with LambdaRank gradients. NDCG is not differentiable, so instead of differentiating it, LambdaRank defines for every pair (i, j) in a query with $rel_i > rel_j$ a gradient whose size is the change in NDCG that swapping them would cause:

$$\lambda_{ij} = -\sigma / (1 + e^{\sigma (s_i - s_j)}) \cdot |\Delta NDCG_{ij}|, \quad \lambda_i = \sum_j \lambda_{ij} - \sum_j \lambda_{ji}$$

Each boosting round fits a regression tree to the λ values (the pseudo-gradients), so every tree pushes files that are wrongly ordered near the top harder than files wrongly ordered near the bottom. XGBoost implements this as the *rank:ndcg* objective with the query structure supplied through *qid*.

Training the ranker (sketch)

```
import xgboost as xgb

# one query per (repository, snapshot)
df["qid"] = df["repo"].astype("category").cat.codes * 100 + df["snapshot_idx"]
df = df.sort_values("qid")                  # XGBoost needs rows grouped by qid

ranker = xgb.XGBRanker(
    objective="rank:ndcg", lambdarank_pair_method="topk", lambdarank_num_pair_per_sample=10,
    n_estimators=2000, learning_rate=0.05, max_depth=6, min_child_weight=5,
    subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0,
    eval_metric=["ndcg@10", "ndcg@20"], early_stopping_rounds=100, tree_method="hist",
)
ranker.fit(X_train, y_train, qid=qid_train, eval_set=[(X_val, y_val)], eval_qid=[qid_val])
scores = ranker.predict(X_test)          # higher = rank earlier
```

6.4 Hyperparameter search

Optuna (tree-structured Parzen estimator) searches max_depth (3 to 10), learning_rate (0.01 to 0.2), min_child_weight (1 to 20), subsample and colsample_bytree (0.5 to 1.0), reg_lambda and reg_alpha (log scale) and the decay half-life (30 to 365 days). The objective is mean NDCG@10 on the validation folds of the grouped temporal split. Early stopping chooses the number of trees.

6.5 Optional monotone constraints

Trust improves when the model behaves sensibly: all else equal, more recent churn or more prior bugs should never *lower* a file's priority. XGBoost supports *monotone_constraints* per feature. Constraining a few features (decayed churn, prior latent defects, transitive dependents) costs little accuracy and removes counter-intuitive explanations. The effect is measured, not assumed.

6.6 Baselines

A learned model is only worth its complexity if it beats simple rules. Every experiment reports:

Baseline	Ranking rule	Why include it
Random	random order (mean of 100 shuffles)	Floor; checks the metric code
LOC	largest files first	Famously strong in defect prediction; any model must beat it
Complexity	highest total cyclomatic complexity first	What a plain static analyser offers
Hotspot	percentile(churn) x percentile(complexity)	The industry heuristic (CodeScene style)
Prior bugs	most prior latent defects first	"Bugs predict bugs" rule
Pointwise RF	Random Forest classifier probability	The original plan; shows the value of ranking
Pointwise XGB	XGBoost regressor on P	Same features, different objective

Table 5. Baselines reported alongside the learned ranker.

7. Feature Selection and Model Hygiene

- **Correlated features.** SLOC, total complexity and number of functions are strongly correlated. Trees cope with this, but explanations split credit among them. Features with Spearman correlation above 0.9 are clustered, and one representative per cluster is kept for the explanation layer (the model may keep all of them).
- **Missing values.** Files with no history (new at T) have undefined trends; XGBoost's native missing-value handling learns a default branch, and an *is_new* flag is added.
- **Class imbalance.** Most files have grade 0. Listwise ranking is naturally robust to this, and queries with no positive file are dropped from training because they provide no ordering signal.
- **Leakage tests.** An automated test re-computes features for a sample of rows from a repository checked out at T and asserts they are identical to the stored ones. Any feature that looks at the future fails this test.
- **Label-shuffle test.** Training on shuffled labels must give NDCG close to the random baseline. If it does not, the pipeline is leaking.

8. Evaluation Protocol

8.1 Two splits, both honest

Random row-level splits would put files from the same repository and the same period in both training and test data, which inflates every metric. TDP uses two stricter protocols:

- **Temporal split (within-project):** train on early snapshots, test on the latest one. This matches real use: the model is trained on the past and applied to the present.
- **Leave-one-project-out (cross-project):** train on all repositories except one, test on that one, and repeat for each repository. This matches the product's main use case: analysing a repository the model has never seen.

A temporal split also needs an **embargo**: a training row at snapshot T_k has a label window ending at $T_k + H$, which must not extend past the test snapshot. Otherwise the training labels would contain information from the test period.

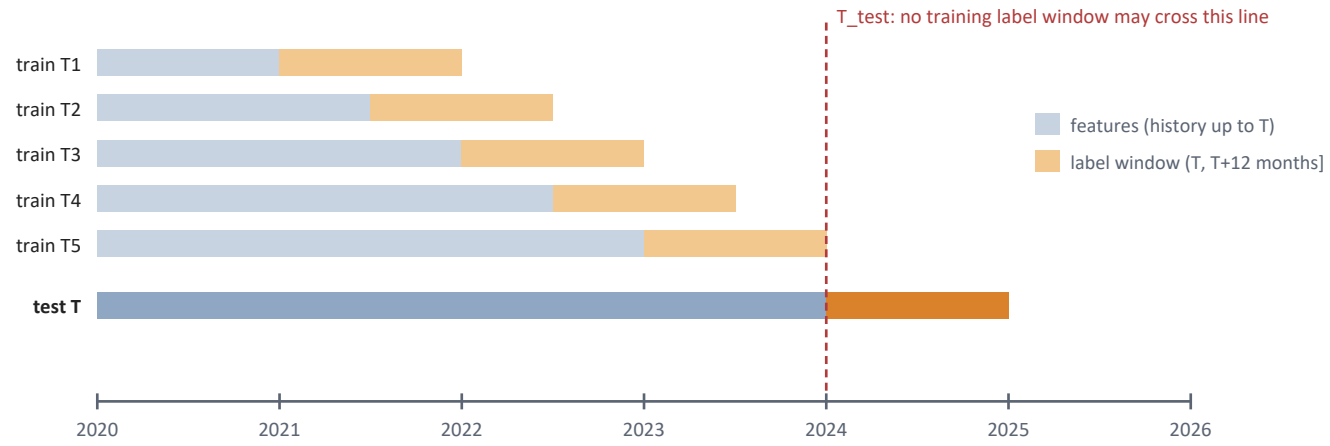


Figure 4. Temporal split with embargo. Snapshots every 6 months, horizon H = 12 months. The last training snapshot (T5) has its label window end exactly at the test snapshot.

Within the training data, hyperparameters are tuned with **grouped, time-ordered folds** (forward chaining: train on T1 to T3, validate on T4; train on T1 to T4, validate on T5).

8.2 Metrics

Metric	Definition	Question it answers
NDCG@10, NDCG@20	Section 6.2, per query, averaged	Is the top of the list right?
Precision@k	share of the top k files with grade 3 or more	If we fix the top 10, how many were real problems?
Recall@20% LOC	share of future latent defects in the files covering the first 20% of LOC, in ranked order	How much risk do we remove with a fixed amount of reading?
P_{opt}	1 - ($Area_{opt}$ - $Area_{model}$) / ($Area_{opt}$ - $Area_{worst}$) on the LOC-based lift chart	Effort-aware ranking quality (Mende and Koschke, 2010)
Spearman ρ	rank correlation between score and P	Does the whole ordering agree, not just the top?
AUC-PR	for the binary has_latent_defect	Classifier view, robust to imbalance
Brier score, ECE	squared error and expected calibration error of the probability	Can the 64% be trusted?

Table 6. Evaluation metrics.

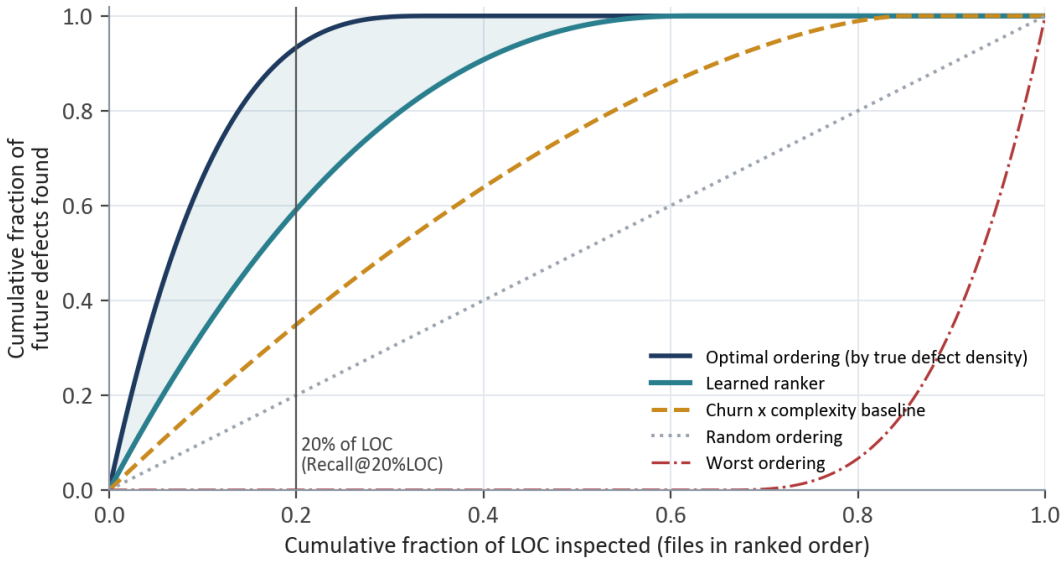


Figure 5. Effort-aware lift chart (illustrative). Files are inspected in ranked order; the x-axis is the share of total LOC read so far. P_{opt} measures how close the model curve is to the optimal curve. The shaded gap is what remains to be gained.

Why effort-aware metrics matter

A model that ranks large files first can score well on accuracy and AUC simply because large files contain more bugs, while being useless in practice: reviewing them takes the whole budget. Recall@20% LOC and P_{opt} charge the model for the size of what it recommends.

8.3 Statistical comparison

Results are per repository, so comparisons use paired tests across repositories: the **Wilcoxon signed-rank test** (model vs each baseline, Holm-corrected) and **Cliff's delta** for effect size (negligible below 0.147, small below 0.33, medium below 0.474, otherwise large). Scott-Knott ESD ranking of all methods is reported for an overall picture (Tantithamthavorn et al., 2017). Mean and 95% bootstrap confidence intervals accompany every headline number.

8.4 Ablation study

The novelty claim, that repository evolution and dependency impact improve prioritisation over static metrics, is tested directly by adding feature families one at a time and measuring the change in each metric:

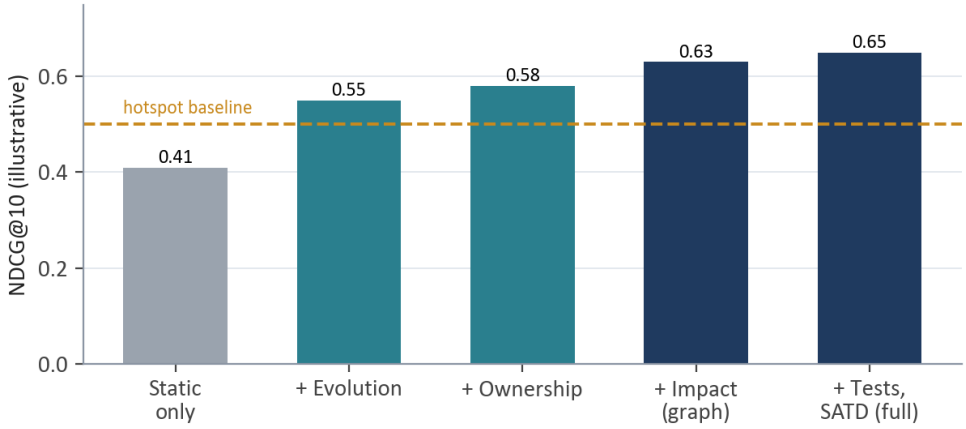


Figure 6. Shape of the ablation result (illustrative values). Each bar adds one family to the previous model. The real numbers are produced by the protocol above.

A second ablation **removes** one family at a time from the full model (leave-one-family-out). Together the two views show both what each family adds alone and what is unique to it.

8.5 External sanity check with PROMISE

The PROMISE repository's defect datasets (for example the Jureczko and Madeyski Java projects with CK metrics) use a different feature set and file-level binary labels, so they are not merged into training. Instead, the static-feature part of the pipeline is evaluated on them as a sanity check: a model restricted to comparable static metrics should reach AUC values in the range reported in the literature. A large gap would point to a bug in the metric extractors.

9. Calibration: Giving the Score a Meaning

A ranking score has no units: 2.45 means nothing to a developer. TDP reports two numbers with clear meanings.

9.1 Priority (0 to 100): a relative position

$$Priority_f = round(100 \cdot percentile\ of\ s(x_f)\ among\ files\ in\ the\ same\ repository)$$

"Priority 92" means the file ranks above 92% of the files in its repository. Severity bands are fixed on this scale: **Critical** at 90 or above, **High** 75 to 89, **Medium** 50 to 74, **Low** below 50. Because bands are relative, every repository has some Critical files; the risk probability below shows whether those files are risky in absolute terms.

9.2 Risk (0 to 100%): an absolute probability

An isotonic-regression calibrator maps the ranking score to the probability that the file has a latent defect fixed within the horizon. It is fitted on held-out folds (never on the data the ranker was trained on). Isotonic regression fits a non-decreasing step function, which preserves the ranking and fixes the shape of the probabilities.

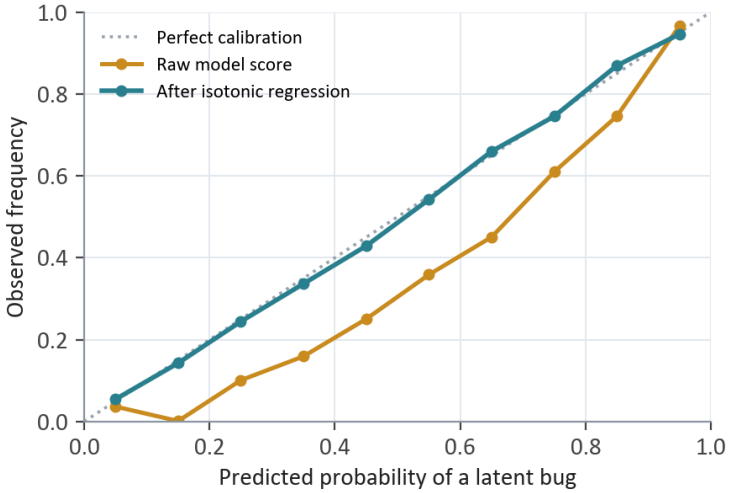


Figure 7. Reliability diagram (illustrative). After calibration, files predicted at 60% have a latent defect fixed about 60% of the time.

Calibration is evaluated with the **Brier score** and **expected calibration error** (ECE, 10 bins). The dashboard shows the probability with a short range from the bootstrap (for example "64%, range 55 to 71%") so that uncertainty is visible.

10. Explainability with SHAP

10.1 Shapley values

For a model *f* and an input *x* with feature set *N*, the Shapley value of feature *i* is its average marginal contribution over all orders in which features could be added:

$$\varphi_i = \sum_{S\ :\ subsets\ of\ N\ without\ i} |S|!\,(|N| - |S| - 1)!\,/\,|N|!\cdot [\,f(S\ plus\ i) - f(S)\,]$$

Shapley values are the only attribution that satisfies *local accuracy* (the contributions add up to the prediction minus the average prediction), *missingness* and *consistency* (Lundberg and Lee, 2017). Computing them naively is exponential, but **TreeSHAP** computes them exactly for tree ensembles in time polynomial in tree depth and number of leaves (Lundberg et al., 2020). For the ranker, SHAP explains the ranking margin *s(x)*.

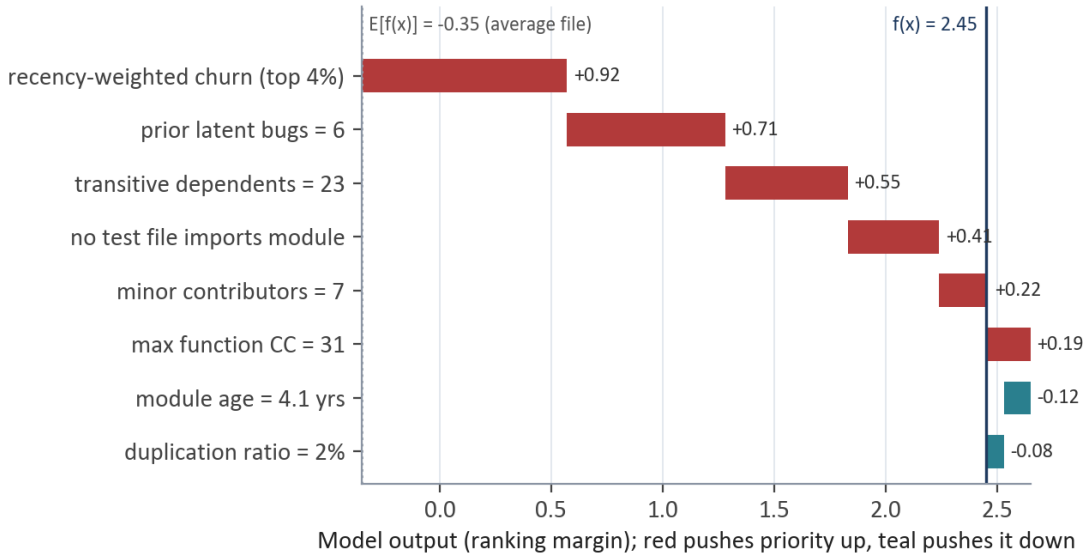


Figure 8. SHAP waterfall for one file (illustrative). Contributions start from the average file's score and add up to this file's score.

10.2 From SHAP values to readable reasons

1. **Group by family.** Sum ϕ within each feature family (Section 4). Correlated features split credit among themselves, but their family total is stable, so reason categories are robust.
2. **Pick reasons.** Take the families with the largest positive totals (up to 4) and, inside each, the feature with the largest ϕ.
3. **Fill a template** with the raw value and its percentile: *"Changes very often: 47 commits in the last 6 months (top 4% in repo)"*. Each feature has a template and thresholds for the adjective ("often", "very often").
4. **Show counter-reasons** (largest negative ϕ) when they are large, for example *"Mitigating: well tested (3 test files import it)"*.

10.3 Global explanations

Mean |ϕ| per feature and per family across the test set gives global importance. SHAP dependence plots (feature value against its ϕ) show the learned shape, for example whether risk rises steadily with transitive dependents or saturates. These plots go in the report and help catch nonsensical behaviour.

11. Debt Types and SATD Classification

11.1 Debt-type assignment

A score says *how urgent*; a debt type says *what kind of fix*. Each prioritised file gets one or two types from its dominant SHAP families plus simple thresholds:

Debt type	Triggered by	Typical remedy
Complexity debt	family S dominant, max CC above 15 or nesting above 4	extract functions, simplify conditionals
Duplication debt	duplication ratio above 10%	extract shared helper, consolidate clones
Test debt	family T dominant, no test importers, low test co-change	characterisation tests before any refactor
Architecture debt	SCC size above 1, high hidden coupling or betweenness	break cycles, introduce interfaces, move code
Knowledge debt	owner inactive, high author entropy	pair-review, document, assign an owner
Admitted debt	SATD density in top 10%	resolve the TODO/HACK items, starting with the oldest
Defect hotspot	family D and E dominant	root-cause review of past bugs, add regression tests

Table 7. Debt types and the signals that trigger them.

As an optional, data-driven complement, files are clustered in **SHAP space** (k-means on the vector of family contributions, k chosen by silhouette score). Clustering explanations rather than raw features groups files by *why* they are risky, which tends to recover the types above and occasionally reveals new ones.

11.2 SATD detection

Comments are extracted from the syntax tree, so comment-like text inside strings is excluded. Classification is two-stage:

1. **Pattern stage:** comments containing TODO, FIXME, HACK, XXX, KLUDGE, "workaround", "temporary" are candidates with high recall.
2. **Classifier stage:** a TF-IDF (word 1 to 2-grams) + logistic regression model decides whether a comment admits debt ("TODO: hack until upstream fixes #881") or is just a task note ("TODO: add docstring"). Training data: the public SATD dataset of Maldonado, Shihab and Tsantalis (2017), about 62,000 labelled comments from ten Java projects, plus a few hundred hand-labelled Python comments for domain adaptation. Evaluation: precision, recall and F1 on a held-out project.

A fine-tuned small transformer (for example a CodeBERT-style encoder) is a stretch goal; it usually improves F1 by a few points but needs a GPU to train.

12. Effort-Aware Prioritisation and the Fix Plan

12.1 Expected benefit

For each file, the expected benefit of fixing it combines the calibrated risk with how far a failure would spread:

$$benefit_f = p_f \cdot I_f \qquad I_f = 1 + \log_2(1 + \text{transitive dependents}_f)$$

The logarithm keeps a module with 500 dependents from overwhelming everything else while still preferring widely used code. The impact formula can be replaced with a learned one when real incident data exists.

12.2 Remediation cost

Cost is estimated in developer-hours with a transparent, SQALE-style model (Letouzey, 2012): every detected issue has a remediation function, and the costs add up:

Issue	Remediation function (default, configurable)
Excess complexity	10 min per complexity point above 10, per function
Duplication	30 min per clone group + 1 min per duplicated line
Missing tests	20 min per public function without a test importer (writing characterisation tests)
Dependency cycle	2 h per module in the strongly connected component
SATD item	30 min per classified SATD comment
Blast-radius overhead	x $(1 + 0.1 \log_2(1 + \text{fan-in}))$: widely used code needs more care and review

Table 8. Default remediation cost model.

Costs are reported as ranges (for example 6 to 9 hours, from 0.8x and 1.2x the point estimate) because they are estimates. Teams can override the constants; the dashboard recomputes the plan immediately.

12.3 Budgeted selection: 0/1 knapsack

Given a budget of *B* hours, choose the set of fixes that maximises the total expected benefit:

$$\text{maximise } \sum_f benefit_f \cdot z_f \quad \text{subject to } \sum_f cost_f \cdot z_f \le B, \; z_f \text{ in } \{0, 1\}$$

With costs rounded to half-hours and a budget of at most a few hundred hours, dynamic programming solves this exactly in O(n · B) time, which is milliseconds for a few thousand files. For very large inputs, the greedy ratio rule (sort by benefit/cost) plus the single best item gives at least half of the optimum and is used as a fallback.

Exact 0/1 knapsack by dynamic programming

```
def plan_fixes(items, budget_hours, step=0.5):
    # items: list of (file_id, benefit, cost_hours)
    W = int(budget_hours / step)
    w = [max(1, round(c / step)) for _, _, c in items]
    best = [0.0] * (W + 1)                    # best[c] = max benefit using capacity c
    keep = [[False] * (W + 1) for _ in items]
    for i, (_, benefit, _) in enumerate(items):
        for c in range(W, w[i] - 1, -1):         # iterate capacity downwards: each item used once
            cand = best[c - w[i]] + benefit
            if cand > best[c]:
                best[c] = cand
                keep[i][c] = True
    chosen, c = [], W
    for i in range(len(items) - 1, -1, -1):       # walk back to recover the chosen set
        if keep[i][c]:
            chosen.append(items[i][0])
            c -= w[i]
    return chosen, best[W]
```

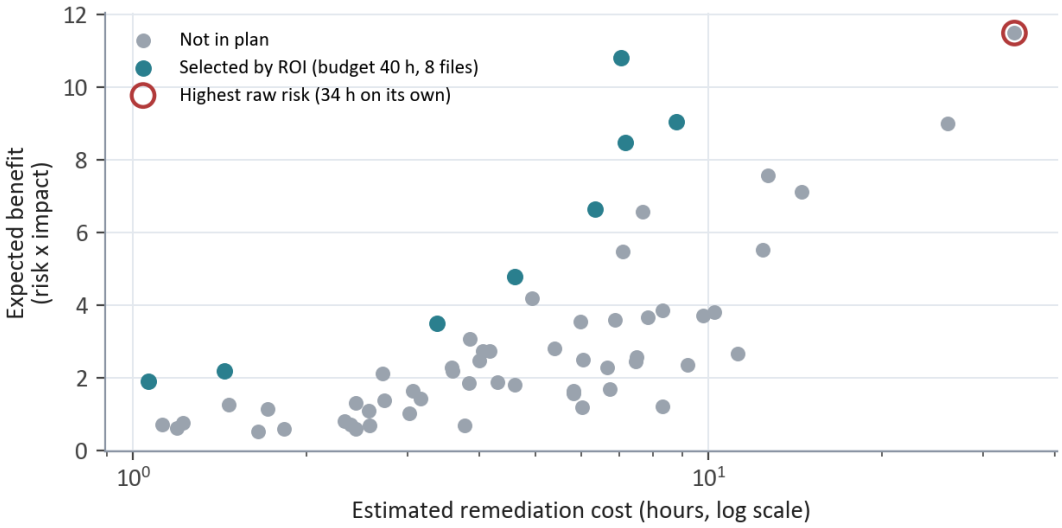


Figure 9. Why ROI matters (illustrative). The file with the highest raw risk (circled) would use 34 of the 40 hours by itself. The knapsack plan fixes 8 cheaper files and removes much more expected risk in total.

12.4 Ordering the plan: dependency-aware scheduling

The order of fixes matters: refactoring a low-level module after the modules that depend on it means touching those dependents twice. The scheduler:

1. Takes the sub-graph of G_{imp} induced by the chosen files.
2. Condenses strongly connected components (Tarjan's algorithm) into single nodes, since files in a cycle must be fixed together. This also breaks the cycle as part of the fix.
3. Topologically sorts the condensed graph so that dependencies come before their dependents.
4. Breaks ties with ROI: among nodes whose prerequisites are done, the highest benefit/cost goes first (Kahn's algorithm with a priority queue).

The result is the "recommended fix order" with a reason for each position, for example "step 1: fix before billing/invoice.py, which depends on it".

12.5 What-if simulator

The dashboard lets a user change actionable features of a file and see the new score: "if this module had a test file" sets the test features to their values for tested files; "if the longest function were split" lowers maximum complexity; "if duplication were removed" sets the ratio to zero. The model is re-run on the modified vector and the new priority, risk and plan are shown.

> **Limits of what-if analysis**
> This is a model counterfactual, not a causal estimate. It answers "how would the model see this file", which is useful for understanding and for motivating fixes, but the report must not claim that adding a test *causes* a 30% drop in bugs. Only non-actionable features (age, history) are frozen.

13. Generative AI Layer: Grounded Fix Advice

The ranker decides *what* to fix. A large language model helps with *how*, for the top files only and only when the user asks.

13.1 Inputs (grounding)

- File path, language, priority, risk, debt types and the reason list from Section 10.
- The 1 to 3 most complex functions, located by Tree-sitter and trimmed to a token budget (about 300 lines total).
- Direct dependents and dependencies (names only), test files that import the module, SATD comments.
- Strict instructions: use only the provided facts, refer only to functions that appear in the excerpt, and answer in a fixed JSON schema.

13.2 Output schema

Structured recommendation returned by the LLM

```
{
  "summary": "Why this file is risky, in 2-3 sentences, based only on the given reasons",
  "steps": [
    {"order": 1, "action": "Add characterisation tests for charge() and refund()",
     "targets": ["charge", "refund"], "effort_hours": [2, 3]},
    {"order": 2, "action": "Extract retry and logging from charge() into _with_retry()",
     "targets": ["charge"], "effort_hours": [1, 2]}
  ],
  "tests_to_add": ["refund() with a partially captured payment", "..."],
  "risks": ["billing/invoice.py relies on the current exception types"],
  "confidence": "medium"
}
```

13.3 Validation (no unchecked text reaches the user)

1. Parse with a Pydantic model; on failure, retry once with the validation error appended, then give up gracefully.
2. Check that every name in *targets* exists in the file's syntax tree; drop or flag steps that refer to functions that do not exist (hallucination guard).
3. Clip effort estimates to the range of the cost model in Section 12 and show both.
4. Label the panel as AI-generated advice to review, never as a verified fix.

13.4 Model choice, cost and privacy

A current mid-size model (for example Claude Sonnet 5.5, model id *claude-sonnet-5-5*) gives good refactoring advice at moderate cost; a small model (Claude Haiku 5.5) is enough for SATD summaries. Responses are cached by (file content hash, model version, prompt version). For private repositories the feature is off by default, and only the trimmed excerpt is sent, never the whole repository.

13.5 Evaluating the advice

Generated advice is evaluated on a sample of 30 to 50 top-ranked files with a rubric (correctness, specificity, actionability, grounding, each 1 to 5) scored by the team, plus the automatic hallucination rate (share of steps naming non-existent functions). An LLM-as-judge pass with a different model can scale this, provided it is checked against the human scores first.

14. Learning from Feedback

Users can mark any ranked file as *agree*, *disagree: not a problem* or *disagree: should be higher*. Each judgement becomes a pairwise preference inside that repository (for example, file A should rank above file B). These preferences are used in two ways:

- **Per-repository adjustment:** a small re-ranking step learns a correction on top of the global model for that repository (a pairwise logistic model on SHAP family totals), so a team's priorities are respected without retraining the global model.
- **Active learning:** the dashboard asks for feedback on files where the model is least certain (scores near the Critical/High boundary, or high variance across bootstrap models), which gives the most information per click.

Feedback is also an evaluation signal: agreement rate on the top 10 is tracked per model version.

15. Extensions and Threats to Validity

15.1 Extensions

- **Function-level granularity:** needs tracking functions across renames and moves (for example by AST similarity), which is a project of its own. File-level results come first.
- **More languages:** Java next (same Tree-sitter pipeline, a new import resolver), then TypeScript.
- **Survival analysis:** model time until the next latent defect is fixed (Cox model or gradient-boosted survival) instead of a fixed horizon.
- **Graph neural networks** on the dependency graph: interesting, but rarely better than boosted trees on tabular features at this dataset size, so left for future work.

15.2 Threats to validity

Threat	Effect	Mitigation
SZZ and bug-fix detection noise	wrong labels	issue-linked fixes preferred, manual precision check, sensitivity analysis
Tangled commits	unrelated files credited with a bug	SZZ line-level tracing, bulk-commit filter
Unreported bugs	risky files look clean	only repos with active issue trackers; ranking is relative within a repo
Project selection bias	results may not transfer	diverse projects, leave-one-project-out evaluation
Label definition choice	conclusions depend on γ and H	sensitivity analysis reported
Static test proxies	imperfect coverage signal	validated against real coverage on demo repos

Table 9. Threats to validity and mitigations.

16. References

[1] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, 1976.

[2] N. Nagappan and T. Ball. Use of relative code churn measures to predict system defect density. *ICSE*, 2005.

[3] C. Bird, N. Nagappan, B. Murphy, H. Gall and P. Devanbu. Don't touch my code! Examining the effects of ownership on software quality. *ESEC/FSE*, 2011.

[4] J. Sliwerski, T. Zimmermann and A. Zeller. When do changes induce fixes? *MSR*, 2005.

[5] S. Kim, T. Zimmermann, K. Pan and E. J. Whitehead. Automatic identification of bug-introducing changes. *ASE*, 2006.

[6] H. Gall, K. Hajek and M. Jazayeri. Detection of logical coupling based on product release history. *ICSM*, 1998.

[7] T. Mende and R. Koschke. Effort-aware defect prediction models. *CSMR*, 2010.

[8] Y. Kamei et al. A large-scale empirical study of just-in-time quality assurance. *IEEE TSE*, 2013.

[9] T. Zimmermann, N. Nagappan, H. Gall, E. Giger and B. Murphy. Cross-project defect prediction. *ESEC/FSE*, 2009.

[10] C. J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Microsoft Research Technical Report MSR-TR-2010-82, 2010.

[11] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. *KDD*, 2016.

[12] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. *NeurIPS*, 2017.

[13] S. M. Lundberg et al. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2020.

[14] A. Potdar and E. Shihab. An exploratory study on self-admitted technical debt. *ICSME*, 2014.

[15] E. Maldonado, E. Shihab and N. Tsantalis. Using natural language processing to automatically detect self-admitted technical debt. *IEEE TSE*, 2017.

[16] J.-L. Letouzey. The SQALE method for evaluating technical debt. *MTD Workshop*, 2012.

[17] C. Tantithamthavorn, S. McIntosh, A. E. Hassan and K. Matsumoto. An empirical comparison of model validation techniques for defect prediction models. *IEEE TSE*, 2017.

[18] D. Spadini, M. Aniche and A. Bacchelli. PyDriller: Python framework for mining software repositories. *ESEC/FSE*, 2018.

[19] S. Schleimer, D. S. Wilkerson and A. Aiken. Winnowing: local algorithms for document fingerprinting. *SIGMOD*, 2003.

[20] L. Page, S. Brin, R. Motwani and T. Winograd. The PageRank citation ranking: bringing order to the web. Stanford technical report, 1999.

[21] R. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1972.

[22] A. Tornhill. *Your Code as a Crime Scene*. Pragmatic Bookshelf, 2015.

[23] M. Jureczko and L. Madeyski. Towards identifying software project clusters with regard to defect prediction. *PROMISE*, 2010.